PARCIAL FINAL: PROGRAMACIÓN ORIENTADA A OBJETOS EN JAVA

El parcial se desarrollará de forma virtual, por grupos, mediante la solución de un caso empresarial asignado. Cada grupo deberá analizar el problema, diseñar una solución orientada a objetos, implementar el programa en Java, probar su funcionamiento y entregar evidencias que permitan evaluar el uso correcto de los conceptos trabajados durante el curso.

1.1 Metodología de evaluación virtual

La metodología de evaluación será denominada **"Evaluación virtual con defensa técnica del código"**. Cada grupo recibirá un caso diferente, pero todos deberán cumplir los mismos criterios técnicos. La calificación no se basará únicamente en que el programa funcione, sino en que el estudiante demuestre, justifique y explique las decisiones tomadas en el diseño, la implementación y las pruebas del código.

Cada entrega debe reflejar un proceso de construcción propio, organizado y comprensible. Por esta razón, el código deberá estar comentado con lenguaje claro, explicando decisiones de diseño, validaciones, uso de colecciones, manejo de excepciones e implementación de hilos. Más que presentar un programa terminado, se espera que el grupo evidencie cómo pensó la solución, cómo la estructuró y qué ajustes realizó para que el sistema respondiera al caso asignado.

1.1.1 Evidencias obligatorias para evaluación remota

Evidencia	Propósito de evaluación
Código fuente comentado	Permite revisar si el estudiante comprende la función de clases, métodos, validaciones, colecciones, hilos y excepciones. Los comentarios deben explicar decisiones propias, no repetir literalmente lo que hace cada línea.
Documento de justificación técnica	Debe explicar por qué se crearon las clases, qué responsabilidades tiene cada una, qué estructuras de control se utilizaron y cómo se relaciona el UML con el programa.
Bitácora de desarrollo	Registro breve de pasos realizados, errores encontrados, correcciones aplicadas y decisiones tomadas durante la construcción del programa.
Capturas o video corto de ejecución	Demuestra que el programa compila, ejecuta el menú, valida errores, procesa los datos y evidencia el uso de hilos cuando corresponda.
Sustentación virtual individual	Permite valorar la apropiación del trabajo mediante una explicación técnica del código, la relación entre el diseño y la implementación, y la claridad con la que cada integrante comprende las decisiones desarrolladas por el grupo.

1.1.2 Estrategias para identificar uso no declarado de IA

El uso de herramientas de inteligencia artificial no será evaluado como falta por sí mismo si el estudiante lo declara y demuestra comprensión del resultado. Sin embargo, se considerará uso no declarado o inapropiado cuando el estudiante no pueda explicar el código, existan comentarios genéricos sin relación con la lógica implementada, aparezcan estructuras no vistas en clase sin justificación, haya diferencias fuertes entre el UML y el código, o el programa incluya nombres de variables, librerías o patrones ajenos al nivel esperado.

La revisión remota se apoyará en evidencias técnicas que permitan comprender el proceso seguido por el estudiante: ejecución del proyecto, claridad de los comentarios, coherencia entre el UML y el código, trazabilidad de la bitácora, manejo de errores y consistencia entre lo documentado y lo implementado. Estas evidencias funcionan como huellas del desarrollo y permiten diferenciar una solución construida progresivamente de una entrega generada sin apropiación técnica.

1.1.3 Declaración de autoría y uso de herramientas.

Cada grupo deberá anexar una declaración breve indicando si utilizó o no herramientas de apoyo, incluyendo inteligencia artificial. En caso de haberlas utilizado, deberá especificar para qué se usaron, qué partes fueron modificadas por el grupo y qué decisiones técnicas fueron tomadas por los estudiantes. La falta de coincidencia entre la declaración, el código y la sustentación podrá ser tenida en cuenta como criterio de revisión académica.

1.2 Condiciones generales para todos los grupos

Componente obligatorio	Criterio de evaluación
Diseño orientado a objetos	Mínimo tres clases con responsabilidades claras, atributos privados, constructores y métodos públicos.
UML básico	Diagrama de clases con atributos, métodos, visibilidad y relaciones entre clases.
Colecciones	Uso de List, ArrayList u otra colección para almacenar objetos del sistema.
Estructuras de control	Uso funcional de if, switch, for y while o do-while.
Excepciones	Validación de datos incorrectos mediante throw, try-catch y mensajes claros para el usuario.
Hilos	Implementación de Runnable o Thread para simular procesos concurrentes con start() y, cuando aplique, join().
Documentación y entrega	Javadoc, capturas de ejecución, código fuente organizado y archivo JAR ejecutable.

6.3 Casos asignados por grupo

Grupo	Caso asignado	Reto específico de validación y concurrencia

1	Sistema de reservas de salas de estudio universitarias.	Validar horarios duplicados y simular varias reservas procesándose al mismo tiempo.
2	Sistema de préstamos de equipos audiovisuales.	Validar disponibilidad del equipo y simular entregas concurrentes.
3	Sistema de turnos para atención en biblioteca.	Controlar prioridad del turno y simular atención simultánea por varios funcionarios.
4	Sistema de control de inventario de papelería.	Validar cantidades negativas y simular varias solicitudes de retiro de inventario.
5	Sistema de inscripción a talleres deportivos.	Validar cupos disponibles y simular inscripciones realizadas por varios estudiantes.
6	Sistema de gestión de pedidos en una tienda escolar.	Validar productos inexistentes y simular despacho concurrente de pedidos.
7	Sistema de registro de citas médicas estudiantiles.	Validar especialidad y fecha, y simular atención concurrente de citas.
8	Sistema de control de entregas de trabajos académicos.	Validar fechas límite y simular revisión simultánea de entregas.
9	Sistema de gestión de reportes de mantenimiento.	Validar prioridad y simular técnicos atendiendo reportes en paralelo.
10	Sistema de ventas de boletería para eventos institucionales.	Validar cantidad de boletas y simular compras concurrentes.
11	Sistema de control de rutas de transporte universitario.	Validar cupo por ruta y simular abordajes simultáneos.
12	Sistema de seguimiento de solicitudes administrativas.	Validar tipo de solicitud y simular procesamiento concurrente por dependencias.
13	Sistema de gestión de reservas de laboratorios de informática.	Validar horario, laboratorio y disponibilidad, simulando reservas concurrentes.

1.4 Preguntas de defensa individual del código.

La defensa individual del código se entiende como una explicación técnica del proceso de desarrollo. Cada integrante debe estar en capacidad de describir la función de las clases creadas, la forma en que se aplica la encapsulación, la colección utilizada, el funcionamiento del menú, las validaciones implementadas, el manejo de excepciones, el comportamiento de los hilos y el procedimiento seguido para generar Javadoc y el archivo JAR.

1.5 Alternativas cuando no sea posible evaluar en vivo

Si el tiempo no permite una sustentación sincrónica, la evaluación podrá realizarse de manera asincrónica a partir de las evidencias entregadas. En este escenario, el parcial queda diseñado para que cada archivo enviado hable del proceso realizado: el código muestra la lógica, los comentarios explican las decisiones, el UML representa la estructura,

la bitácora evidencia la evolución del trabajo y las capturas o videos demuestran el funcionamiento del sistema.
Como parte natural del proceso académico, la entrega podrá quedar sujeta a verificaciones posteriores de comprensión y autoría mediante aclaraciones escritas, formularios breves, comentarios técnicos sobre fragmentos del código o ajustes puntuales solicitados sobre la solución presentada. Estas acciones no buscan rehacer el parcial, sino confirmar que el producto enviado corresponde a un aprendizaje real y a una construcción técnica coherente con el nivel del curso.

1.6 Metodologías asincrónicas complementarias

Metodología	Forma de aplicación	Qué permite evaluar
Preguntas diferidas por correo	Se pueden enviar aclaraciones técnicas breves sobre partes concretas del código después de revisar la entrega.	Comprensión real, autoría y relación entre explicación y código.
Formulario individual de verificación	Cada integrante responde preguntas diferentes sobre clases, métodos, excepciones, hilos, UML y decisiones de diseño.	Participación individual y dominio de conceptos.
Reto de modificación posterior	Se propone un pequeño ajuste sobre el código entregado, por ejemplo agregar una validación, cambiar un estado o modificar una opción del menú.	Capacidad de intervenir el propio código y no solo entregar un producto final.
Bitácora comentada de desarrollo	El grupo entrega un registro de pasos, errores, decisiones, pruebas realizadas y ajustes aplicados.	Proceso de construcción, coherencia técnica y trazabilidad del trabajo.
Explicación escrita por fragmentos	El estudiante selecciona fragmentos clave del código y explica qué hacen, por qué los usó y qué pasaría si se modifican.	Comprensión del funcionamiento interno del programa.
Comparación UML-código	El grupo debe explicar cómo cada clase del diagrama se refleja en el código fuente.	Coherencia entre análisis, diseño e implementación.
Declaración de autoría y uso de IA	El grupo declara si utilizó herramientas de apoyo, qué partes fueron sugeridas y qué ajustes propios realizaron.	Transparencia, responsabilidad académica y detección de uso no declarado.

1.7 Criterio de evaluación cuando solo se revise lo enviado.

Cuando la evaluación se realice únicamente sobre los archivos enviados, se valorará la coherencia entre todos los productos entregados: planteamiento del caso, UML, código fuente, comentarios, Javadoc, capturas de ejecución, archivo JAR, bitácora y justificación técnica. El programa funcional será importante, pero no será el único indicador de

desempeño; también se tendrá en cuenta la claridad del proceso, la organización del proyecto, la calidad de las decisiones de programación y la consistencia del estilo de código con los conceptos trabajados en clase.
Cuando existan diferencias importantes entre lo diseñado, lo programado y lo explicado en los anexos, la entrega podrá pasar a una revisión académica complementaria. Esta revisión permitirá contrastar la autoría, el uso responsable de herramientas de apoyo y la comprensión del código sin convertir el proceso en una nueva oportunidad de elaboración, sino en una verificación de la apropiación técnica del trabajo presentado.